AI Studio → Google Cloud Agent Platform Migration

Nov 10, 2026 · @Darshan Kumar Hegde

Overview

AI Studio uses a prepaid credit model — when credits are exhausted, all LLM calls return `402 RESOURCE_EXHAUSTED` and there is no in-place upgrade to postpaid. The only pay-as-you-go path is the **Gemini Enterprise Agent Platform** (formerly Vertex AI), which bills through a GCP billing account.

Migration requires four things:

1. A GCP project with billing enabled and the Agent Platform API active
2. A service account with the Agent Platform User role and a downloaded JSON key
3. Two code changes — `vertexai=True` on `ChatGoogleGenerativeAI` plus `api_version="v1"`
4. Updated model names (`gemini-2.5-flash` and `gemini-2.5-pro` are inaccessible on new GCP projects)

No change to prompt logic, retrieval, or PDF generation is needed.

GCP Project Setup

1. **Create a GCP project** — console.cloud.google.com → New Project. Note the **Project ID** (not the display name — it appears in the top bar and may differ, e.g. `getschemes` vs `GetSchemes`).
2. **Link a billing account** — Console → Billing → Link a billing account. Create one with a credit/debit card if none exists. The project must appear under "Projects linked to this billing account".
3. **Enable the Agent Platform API** — APIs & Services → Library → search **"Agent Platform API"** → Enable. (Searching "Vertex AI" redirects here — this is the same product after rebranding.)

> AI Studio's "Set up prepay" option in its project settings does **not** enable pay-as-you-go billing even when a GCP billing account is attached. A fresh GCP project with the Agent Platform API is required.

IAM and Service Account

1. **Create a service account** — IAM & Admin → Service Accounts → Create service account. Give it a descriptive name (e.g. `gemini-sa`).
2. **Grant the Agent Platform User role** — IAM & Admin → IAM → find the service account → Edit → Add role → search **"Agent Platform User"** (`roles/aiplatform.user`). This role was formerly called "Vertex AI User"; the role ID is unchanged.
3. **Download a JSON key** — Service Accounts → select the SA → Keys tab → Add Key → JSON. Save the downloaded file to `keys/` in the project root.
4. **Protect the key file** — add `keys/` to `.gitignore` to prevent it being committed.

> API keys created in GCP Console and bound to a service account authenticate against the Generative Language API (AI Studio path), not the Agent Platform. For Agent Platform access, a service account JSON key is required — not an API key string.

Code Changes

Two files require changes. The import and all prompt/schema logic stay the same.

`polisense_core/AssessmentPipeline/prompts.py`

Add `import os` at the top. In `make_gemini_flash()`, replace the `ChatGoogleGenerativeAI` construction:

```
# Before
from langchain_google_genai import ChatGoogleGenerativeAI
return ChatGoogleGenerativeAI(
    model=_FLASH_MODEL,
    temperature=0,
    max_retries=2,
).with_structured_output(schema=schema, method="json_schema")

# After
from langchain_google_genai import ChatGoogleGenerativeAI
return ChatGoogleGenerativeAI(
    model=_FLASH_MODEL,
    temperature=0,
    max_retries=2,
    vertexai=True,
    project=os.environ["GOOGLE_CLOUD_PROJECT"],
    location="global",
    api_version="v1",
).with_structured_output(schema=schema, method="json_schema")
```

`polisense_core/IngestionPipeline/llm_extractor.py`

In `initialize_llm_with_config()`, apply the same four parameters:

```
# After
import os
from langchain_google_genai import ChatGoogleGenerativeAI
model = ChatGoogleGenerativeAI(
    model=model_name,
    temperature=temperature,
    max_tokens=max_tokens,
    timeout=timeout,
    max_retries=max_retries,
    vertexai=True,
    project=os.environ["GOOGLE_CLOUD_PROJECT"],
    location="global",
    api_version="v1",
)
```

Key parameters:

Parameter	Value	Reason
`vertexai`	`True`	Routes calls to `aiplatform.googleapis.com` instead of `generativelanguage.googleapis.com`
`location`	`"global"`	All models are available at the global endpoint; regional endpoints (e.g. `us-central1`) return 404 for most Gemini models
`api_version`	`"v1"`	Agent Platform requires `v1`; the library defaults to `v1beta1` which returns 404
`project`	env var	Must match the GCP Project ID exactly (not the display name)

Environment Variables

In `.env.secrets`, remove `GOOGLE_API_KEY` and add:

```
# Google Cloud Agent Platform
GOOGLE_CLOUD_PROJECT=<gcp-project-id>
GOOGLE_APPLICATION_CREDENTIALS=<absolute-path-to-service-account-key.json>
```

`GOOGLE_APPLICATION_CREDENTIALS` must be an **absolute path**. The `langchain_google_genai` library reads it automatically via the Google auth library when `vertexai=True` is set.

> On Windows, use forward slashes or escaped backslashes:
> `C:/Users/name/project/keys/my-sa-key.json`

Model Names

`gemini-2.5-flash` and `gemini-2.5-pro` return 404 on new GCP projects — they are inaccessible regardless of permissions. Use the replacements below.

AI Studio model	Agent Platform replacement	Used in
`gemini-2.5-flash`	`gemini-3.8-flash`	Assessment pipeline — `AssessmentPipeline/prompts.py` (`_FLASH_MODEL`)
`gemini-2.5-pro`	`gemini-4-argon`	Ingestion pipeline — `IngestionPipeline/llm_extractor.py` (default `model_name`)

Other currently accessible models on the global endpoint (verified Nov 2026): `gemini-3.5-flash` , `gemini-3.1-pro-preview` , `gemini-omni-1.2-flash` .